

复数类

我们使用代数表示法来表示复数，即 $a + bi$ ，其中 a 为实部， b 为虚部。

你需要完成一个基本的复数类，其需要满足以下的功能。

构造函数

对于复数类，我们有以下几个最基本的要求：

1. 默认构造函数，默认复数值为 0。

```
complex z; // z = 0;
```

2. 可以用一个数值构造一个复数类。此时虚部为 0，实部为传入的值。

```
complex z1 = 1; // z = 1;
complex z2 = 2.22; // z = 2.22;
complex z3(114.514); // z = 114.514;
```

3. 可以使用两个数值构造一个复数类。此时第一个数值为实部，第二个数值为虚部。

```
complex z1(1, 2); // z = 1 + 2i;
complex z2(2.22, 3.33); // z = 2.22 + 3.33i;
```

4. 拷贝构造和拷贝赋值函数，可以从其他类拷贝构造或拷贝赋值。

```
complex x; // x = 0;
complex y = x; // y = x = 0;
x = y = 2; // x = y = 2;
```

访问实部和虚部

我们需要实现两个函数 `real()`, `imag()`，分别返回复数的实部和虚部的引用。

```
complex z(1, 2);
std::cout << z.real() << "," << z.imag() << std::endl; // 输出 1,2
z.real() = 3;
z.imag() = 4; // z = 3+4i
```

比较函数

我们要求实现比较运算符 `==` 用于判断两个复数是否相等。由于浮点数有精度问题，我们要求两个复数的实部和虚部的差的绝对值都小于 10^{-6} 时，认为两个复数相等。

$$z_1 = a + bi, z_2 = c + di$$

$$z_1 = z_2 \iff |a - c| < 10^{-6} \text{ and } |b - d| < 10^{-6}$$

```
complex a = 1, b = 2, c = 2;
std::cout << int(a == b) << std::endl; // 输出 0
std::cout << int(b == c) << std::endl; // 输出 1
```

四则运算

加减乘除的表现和正常的数学运算一致。还包含一个取负数的运算符 `-`，和一个取共轭复数（虚部相反）的运算符 `~`。

特别地，对于除法，如果除数为 0 ($|a| < 10^{-6}$ and $|b| < 10^{-6}$)，你需要抛出异常 `divided_by_zero`。其定义在下发文件中给出，你需要补充 `what()` 函数的实现。

```
complex a = 1, b = 0;
try {
    a /= b;
} catch(const std::exception &err) {
    // 此处应该输出 (不包含引号) "complex divided by zero!"
    std::cout << err.what();
}
```

输出函数

输出运算符的输出格式为 $[/-]< \text{实部} >[+/-]< \text{虚部} >i$ ，其中实部和虚部都是浮点数，保留 6 位小数。

- 如果实部为负数 ($\text{real} < -10^{-6}$)，则输出 - 号，否则不输出符号。
- 如果虚部为负数 ($\text{imag} < -10^{-6}$)，则输出 - 号，否则输出 + 号。

```
complex a(-2,0);
complex b(0,-2);
complex c(1,-1e-10);
complex d(-1e-10,1);
std::cout << a << std::endl; // 输出 -2.000000+0.000000i
std::cout << b << std::endl; // 输出 0.000000-2.000000i
std::cout << c << std::endl; // 输出 1.000000+0.000000i 用加号，因为它的虚部绝对值在  $10^{-6}$  以内，认为是 0。属于非负数
std::cout << d << std::endl; // 输出 0.000000+1.000000i 实部无符号，因为它的绝对值在  $10^{-6}$  以内，认为是 0。属于非负数
```

类型转换

我们的复数类要支持转化为 bool 类型，当且仅当复数的值为 0 ($|a| < 10^{-6}$ and $|b| < 10^{-6}$) 时，转化为 false，否则转化为 true。

```
complex a; // 默认初始化为 0
if (a) {
    std::cout << "a is not zero!" << std::endl;
} else {
    std::cout << "a is zero!" << std::endl; // 输出 a is zero!
}
```

其他说明

需要特别注意的是，即使你没有实现 **complex** 的部分函数，你也依然需要保留这些函数的声明，否则将无法通过编译，即使这个测试点没有用到这些函数。

如果你提交的代码中全部 CE，并且具体报错信息里面提示 `undefined reference to 'Some_of_your_member_functions_are_missing()'`，说明你提交的代码里缺少了某些成员函数的声明。请与下发模板文件进行比对，补全缺少的成员函数的声明，再次提交即可。

如果你正确地保留了声明，但是没有实现部分功能，那么你只会对应的测试点出现 CE（报错信息是 `undefined reference to xxx`（具体函数）），其他没有用到这些功能的测试点不会受到影响。

模板在下发文件 `complex.hpp` 中。

提示

在下发文件 `complex.hpp` 中，我们提供了一个函数 `sign` 用于判断一个浮点数的符号，你可以直接使用：

```
// 判断 x 的符号，返回 1 表示  $x > 0$ ，返回 0 表示  $x = 0$ ，返回 -1 表示  $x < 0$ 
const double eps = 1e-6;
inline int sign(double x) {
    return (x > eps) - (x < -eps);
}
```

数据范围

Testcases No.	前置 Test	额外考察的内容	分数
1	无	考察所有构造函数、赋值操作、访问实部和虚部	10
2	1	考察单目运算符 $\sim$ （取共轭）和 $-$ （取负数）	10
3	1	考察加法和减法 $+$, $-$, $+=$, $-=$	10
4	1	考察乘法和除法 $*$, $/$, $*=$, $/=$ （无异常处理）	10
5	1	考察类型转换运算符 <code>bool</code>	10
6	1	考察比较运算符 <code>==</code>	10
7	1	考察输出函数（不要求小数位数，只要你的输出与答案误差在 10^{-4} 内即被认为正确）	10
8	7	考察输出函数（你必须严格按照我们的要求）	10
9	2, 3, 4	无（四则运算测试，含异常处理）	10
10	5, 6, 7, 9	无（综合压力测试，含异常处理）	10

附：下发文件 `complex.hpp`

```
#ifndef SJTU_COMPLEX_HPP
#define SJTU_COMPLEX_HPP

#include <iostream> // 输入输出
#include <iomanip> // 控制输出格式
#include <cmath> // 数学函数
#include <stdexcept> // 异常处理
// 你不可以使用任何其他的头文件!

namespace sjtu {

// 异常类，你需要添加一个函数 what() 来返回异常信息。
class divided_by_zero final : public std::exception {
public:
    divided_by_zero() = default;
    ~divided_by_zero() = default;
    // TODO
};

// 用于判断浮点数的符号。
inline int sign(double x) {
    const double eps = 1e-6;
    return (x > eps) - (x < -eps);
}

// 请保留所有的接口声明，即使你没有实现它们。
// 否则可能出现编译错误，无法通过部分测试点。
// TODO 补全类的实现
class complex {
private:
    double a, b; // a + bi
public:

    complex(); // 默认构造函数 0+0i
    complex(double a); // 构造函数 a+0i
    complex(double a, double b); // 构造函数 a+bi

    double &real(); // 返回实部
    double &imag(); // 返回虚部

    complex operator - () const; // 返回负数
    complex operator ~ () const; // 返回共轭复数

    complex operator + (const complex &rhs) const; // 加法
    complex operator - (const complex &rhs) const; // 减法
    complex operator * (const complex &rhs) const; // 乘法
    complex operator / (const complex &rhs) const; // 除法

    complex &operator += (const complex &rhs);
    complex &operator -= (const complex &rhs);
    complex &operator *= (const complex &rhs);
    complex &operator /= (const complex &rhs);

    bool operator == (const complex &rhs) const; // 判断相等

    explicit operator bool() const; // 转换为 bool

    friend std::ostream &operator << (std::ostream &os, const complex &x); // 输出
```

```
};
```

```
}
```

```
#endif
```